COMS4995W30
Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Fall 2026

AI of the Week: The Hidden "Workhorse" of Modern AI



While the tech world obsesses over the "Attention" mechanisms in models like Transformers, the unsung heroes doing the heavy lifting are the fundamental structures we are learning today, namely

Multilayer Perceptrons and **Backpropagation**

Transformers are Powered by MLPs

- The majority of the parameter count actually resides in the feed-forward neural networks
- These stack simple neurons to transform raw text features into highly abstract representations

Non-Linearity is the Secret to "Understanding"

- Real-world data is highly complex and non-linear
- Industry models rely heavily on modern variants of the ReLU activation, such as GELU, to give the network its flexible decision surfaces

AI of the Week: The Hidden "Workhorse" of Modern AI



Scaling via Backprop & Adam

- Training a model with billions of parameters would be impossible without BP
- Stochastic Gradient Descent can be too slow on massive datasets, industry relies on sophisticated optimizers like Adam to safely navigate complex, high-dimensional loss landscapes

Takeaway

The fundamental rule remains the same whether you are *training a 3-layer network on a laptop* or *a trillion-parameter model across Google's TPUs*: you are approximating complex functions, and minimizing loss via the chain rule

Agenda



- Why Neural Networks
- The Neuron & Activation Functions
- Multilayer Perceptron (MLP)
- Training: Loss → Backprop → Optimizers
- Regularization: Making It Generalize
- MLPs in Industry



Neural Network Fundamentals

Why Neural Networks? 



Many real-world problems are non-linear → linear models fail

- Relationships in data are curved, complex, and high-dimensional

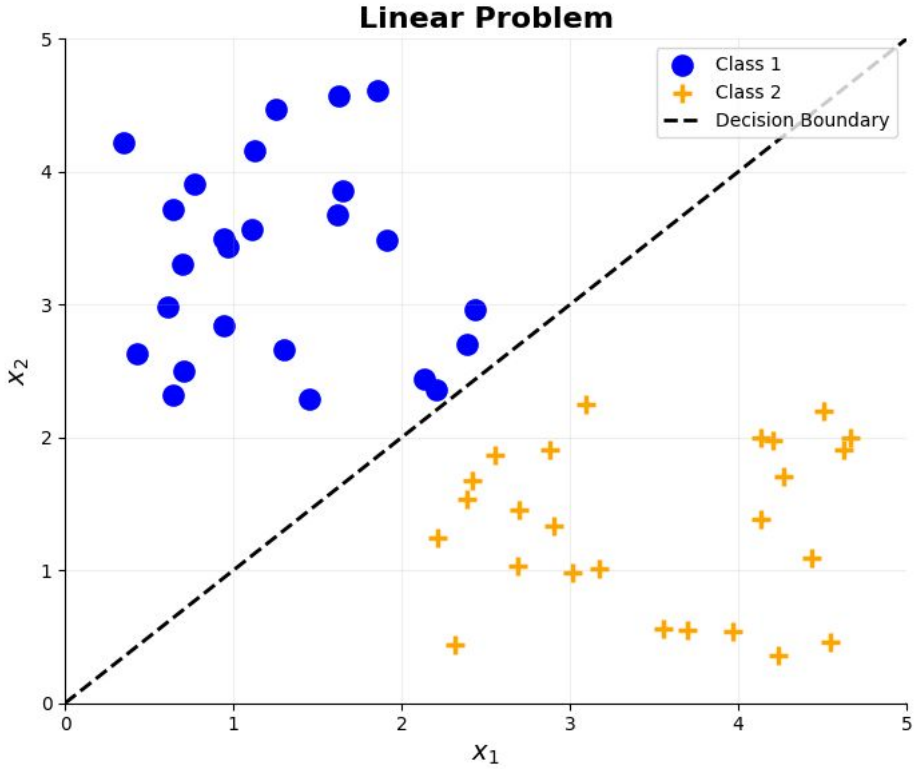
Neural networks can learn complex functions via composition

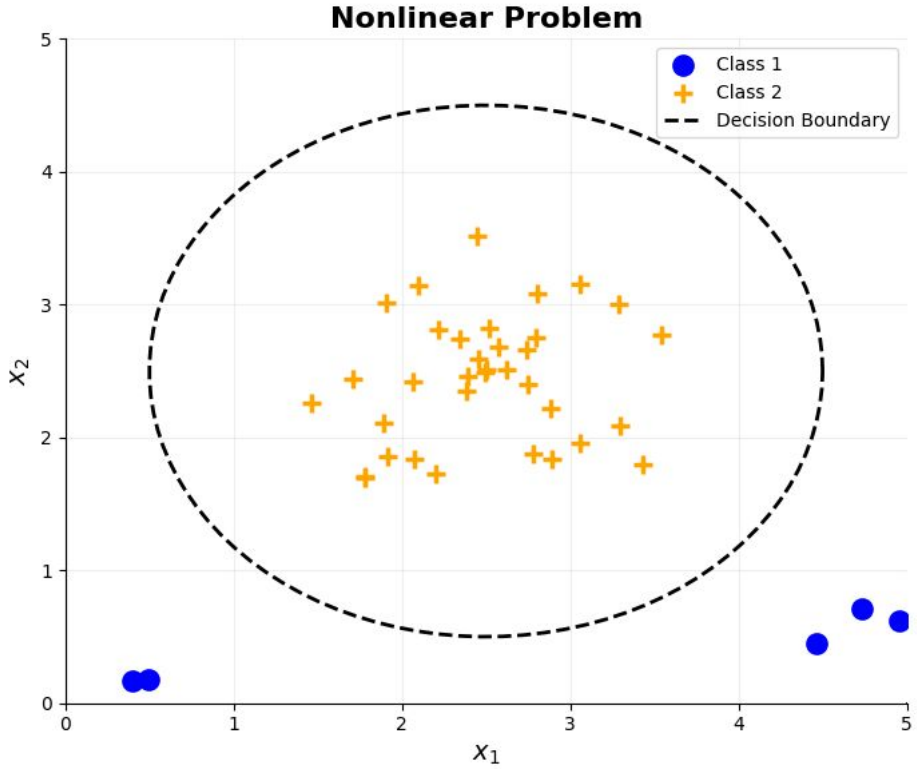
- By stacking neurons, we can approximate any function (Universal Approximation Theorem)

Inspired by the brain: neurons connect and adapt weights

- Each neuron adjusts its connection strength, gradually improving the generalizability

Linear vs Nonlinear





The Big Picture of Neural Networks 🧭



Start from linear models → limited expressiveness

Add nonlinear activations → learn complex functions

Stack layers → Multilayer Perceptron (MLP)

Train by forward + backward passes to minimize loss

Tune learning with optimization and regularization

Forms the foundation for CNNs, Transformers and ChatGPTs

The Big Picture of Neural Networks 🧭



Start from linear models → limited expressiveness

Add nonlinear activations → learn complex functions

Stack layers → Multilayer Perceptron (MLP)

Train by forward + backward passes to minimize loss

Tune learning with optimization and regularization

Forms the foundation for CNNs, Transformers and ChatGPTs

The Neuron ⚙️



Each neuron

- A neuron receives various inputs and computes a weighted sum of them
- This sum is then passed through an activation function, which introduces non-linearity
- The result of this activation becomes the output of the neuron, serving as input for the next layer

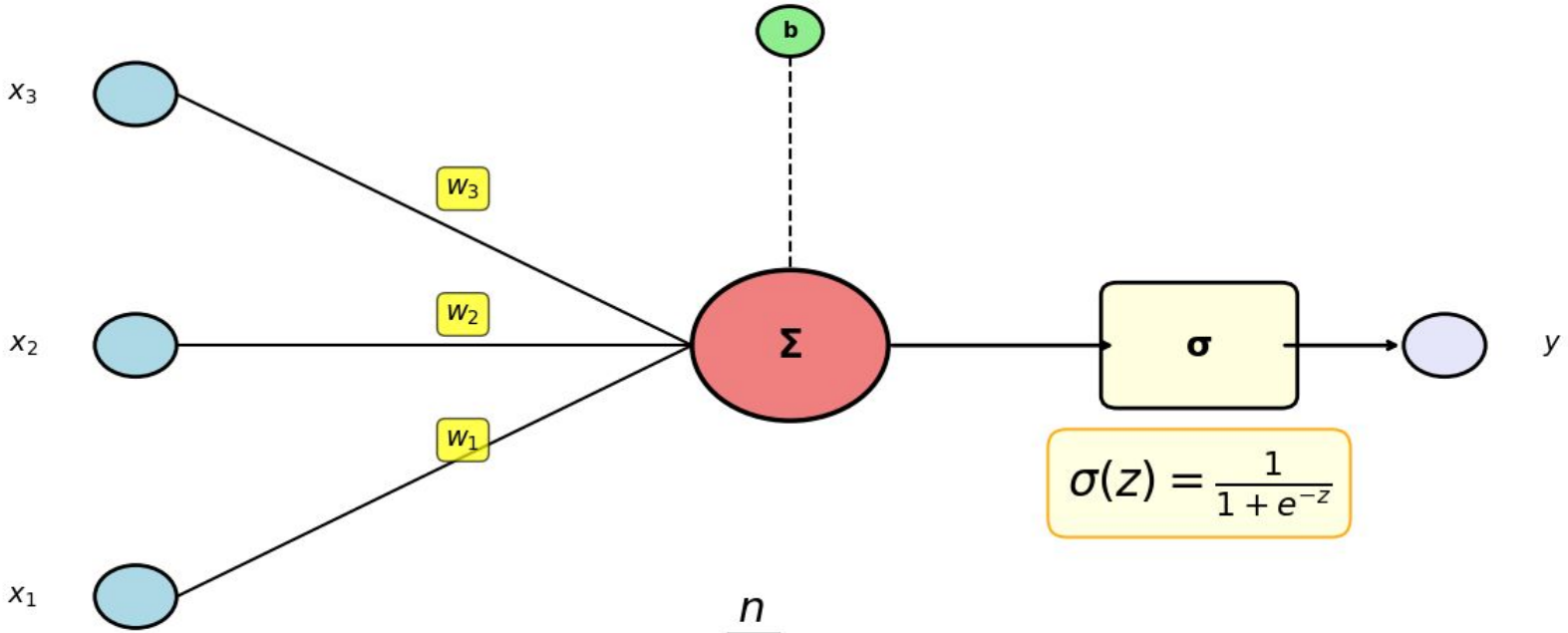
The Neuron ⚙️



Nonlinear activations = flexible decision surfaces

- Without nonlinearity, the entire network would behave like a single linear model
- Nonlinear activations allow the network to learn complex, curved decision boundaries, making it capable of modeling intricate relationships in data

The Neuron 🧠



$$y = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

From Logistic Regression to a Neuron 🔁



You already know one neuron: Logistic Regression (Week 3)

- $z = w \cdot x + b \rightarrow \hat{y} = \sigma(z)$ is exactly the neuron on the previous slide
- Trained the same way: cross-entropy loss + gradient descent

What a neural network adds: learned features

- Week 2: we hand-crafted features (logs, bins, interactions)
- Hidden layers learn those features from the data

A neural network = many logistic regressions, stacked and trained jointly

Activation Functions 🔌



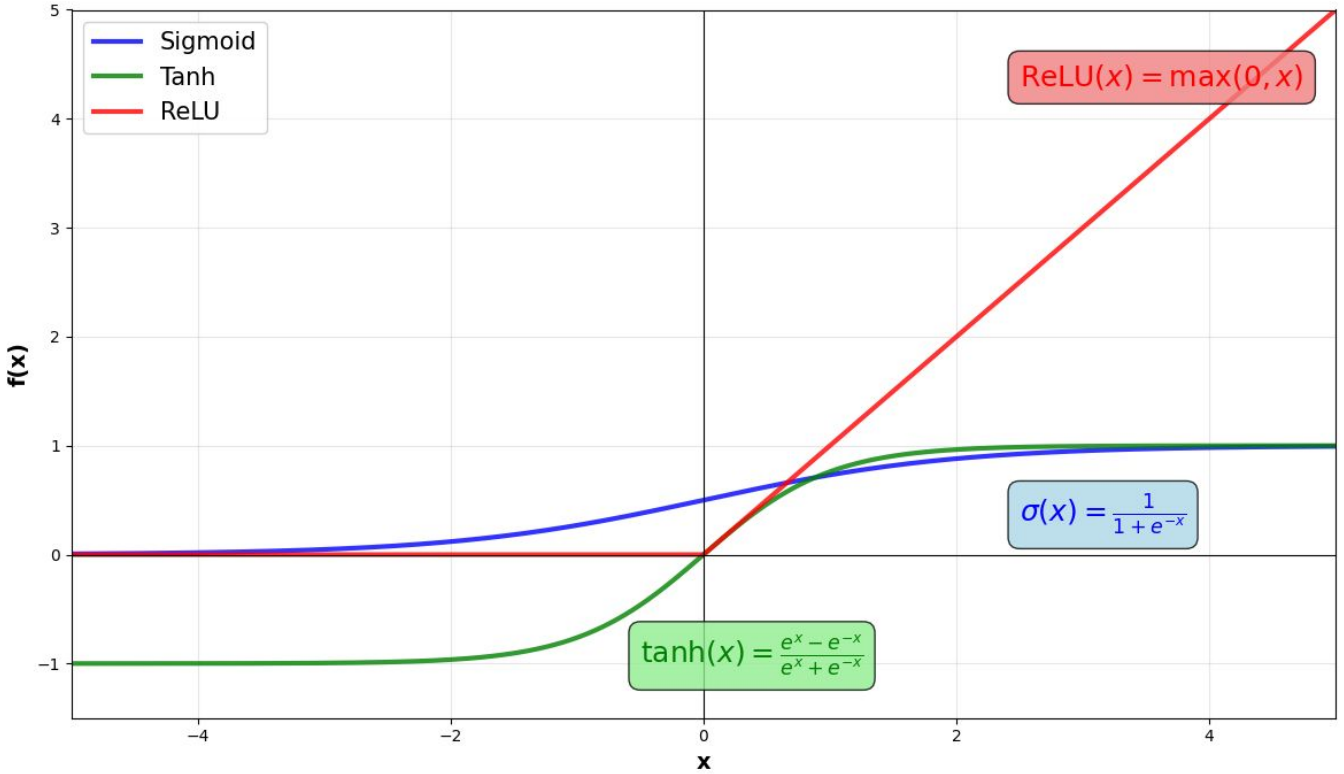
Sigmoid → squashes outputs (good for probabilities)

Tanh → zero-centered, faster convergence

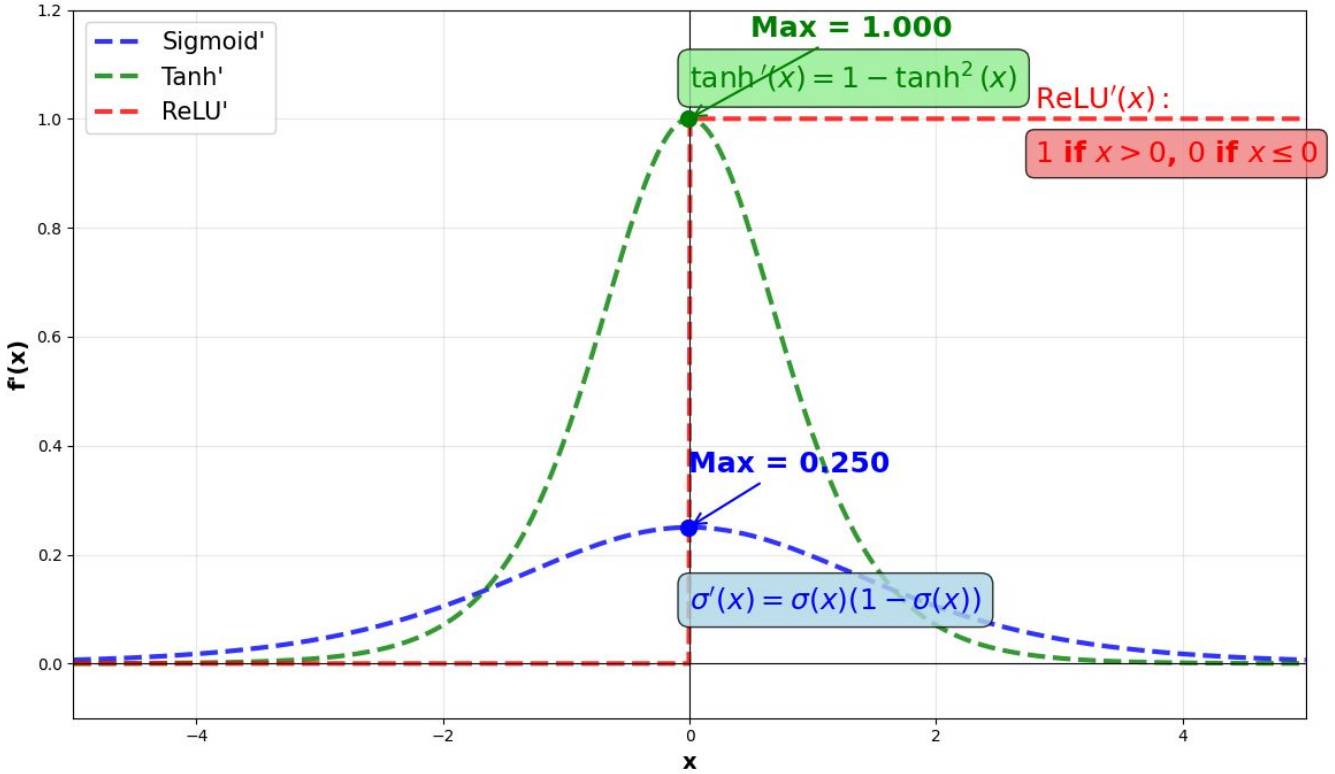
ReLU → sparse activation, efficient & dominant today

Modern variants: Leaky ReLU, GELU (used in Transformers)

Activation Functions Comparison



Derivatives Comparison



🎤 Mock Interview

Interviewer: “A teammate stacks 10 dense layers with no activation functions, only a sigmoid at the output. It scores exactly like logistic regression. Why?”

A) 10 layers is too shallow, add more
B) Without non-linearity, the 10 layers collapse into one linear model
C) The learning rate is too small
D) It needs more training data

30 seconds, no talking. Then raise your hand for one letter.

Multilayer Perceptron (MLP) 🏗️



Stack of Neurons

- A neural network is built by stacking many simple neurons (linear + activation) into layers

Hidden Layers → Feature Learning

- Each hidden layer transforms the data into a new representation - from raw input features to increasingly abstract ones

Multilayer Perceptron (Feed-forward) 🏗️



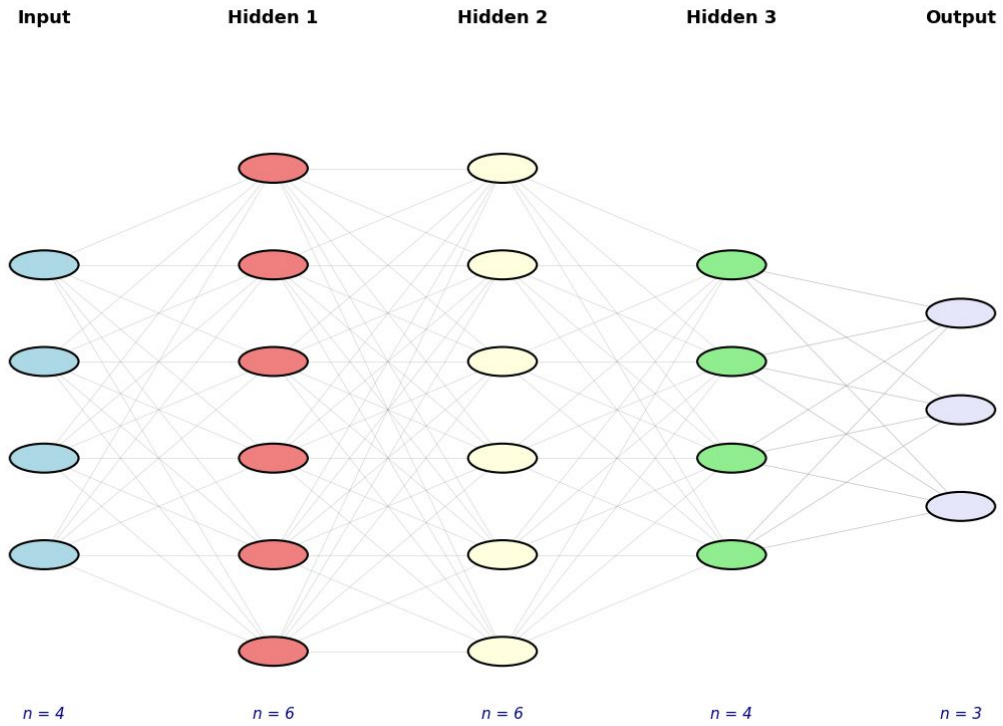
Output Layer → Task-Specific Prediction

- The final layer converts the learned representation into task-relevant outputs:
  - Regression → real values (e.g., price, temperature)
  - Classification → probabilities via Softmax (e.g., cat / dog / car)

🤔 As we go deeper, the network is not simply memorizing

- It automatically represents data using more abstract and meaningful features

Feedforward Architecture



Counting Parameters 🔢

Dense layer = inputs × outputs weights + outputs biases

MNIST MLP: 784 → 256 → 128 → 10

- Layer 1: 784 × 256 + 256 = 200,960
- Layers 2–3: 32,896 + 1,290
- Total ≈ 235K parameters, 85% of them in the first layer

One Transformer MLP block (d = 4,096, hidden 4× wider)

- 4,096 × 16,384 × 2 ≈ 134M parameters, in a single layer
- Most of an LLM's parameters live in these MLPs (AI of the Week)

Training Feedforward Network 🧠



Feedforward (left → right)

- Compute layer activations

Loss function (Distance between prediction and ground-truth)

- Map distance to a scalar error

Backprop (right → left)

- Use the computational graph and chain rule to compute gradients

Training Feedforward Network 🧠



Feedforward (left → right)

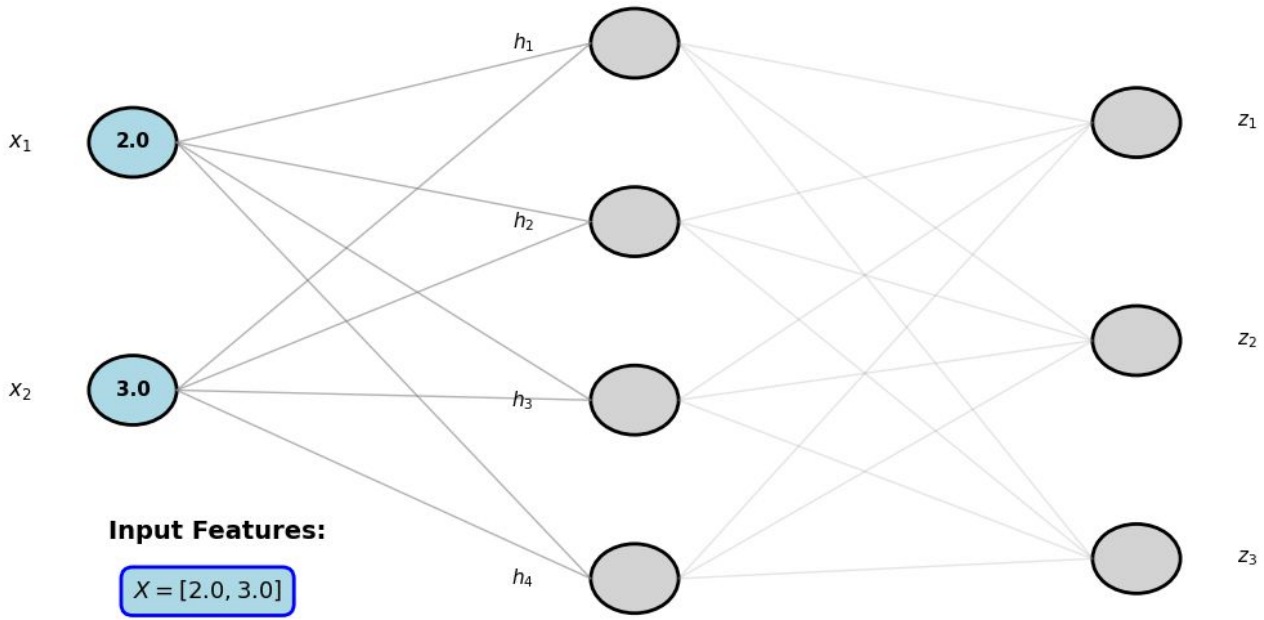
- Compute layer activations

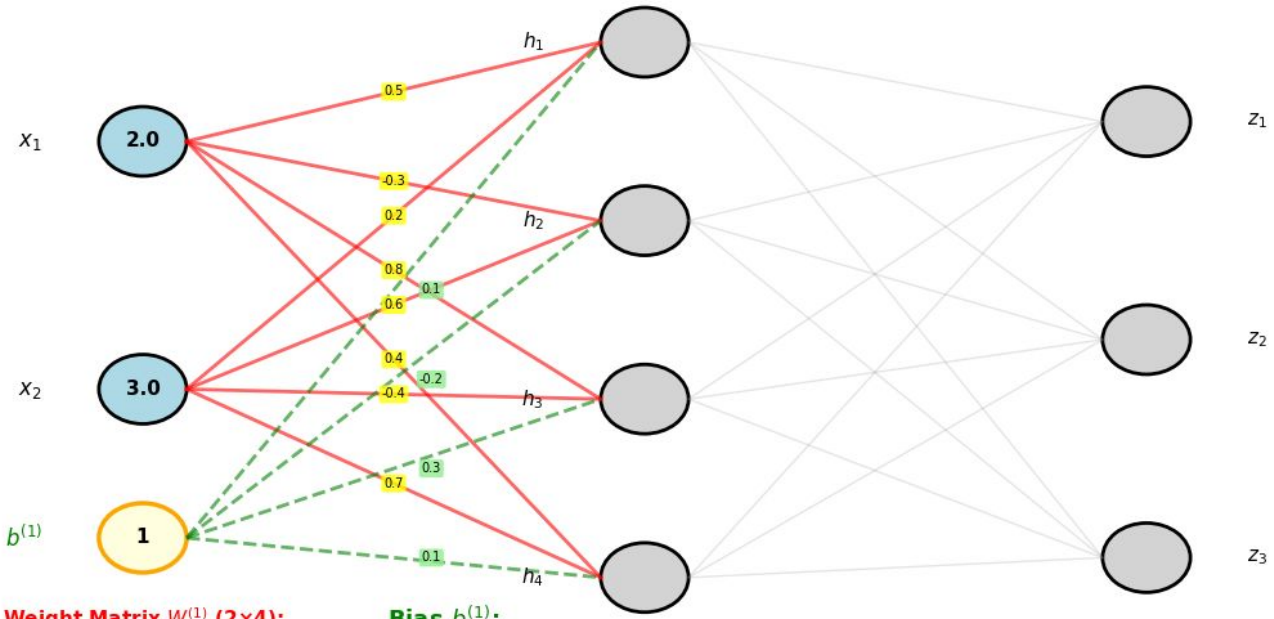
Loss function (Distance between prediction and ground-truth)

- Map distance to a scalar error

Backprop (right → left)

- Use the computational graph and chain rule to compute gradients


x_1
2.0
x_2
3.0
h_1
h_2
h_3
h_4
z_1
z_2
z_3
Input Features:
$X = [2.0, 3.0]$

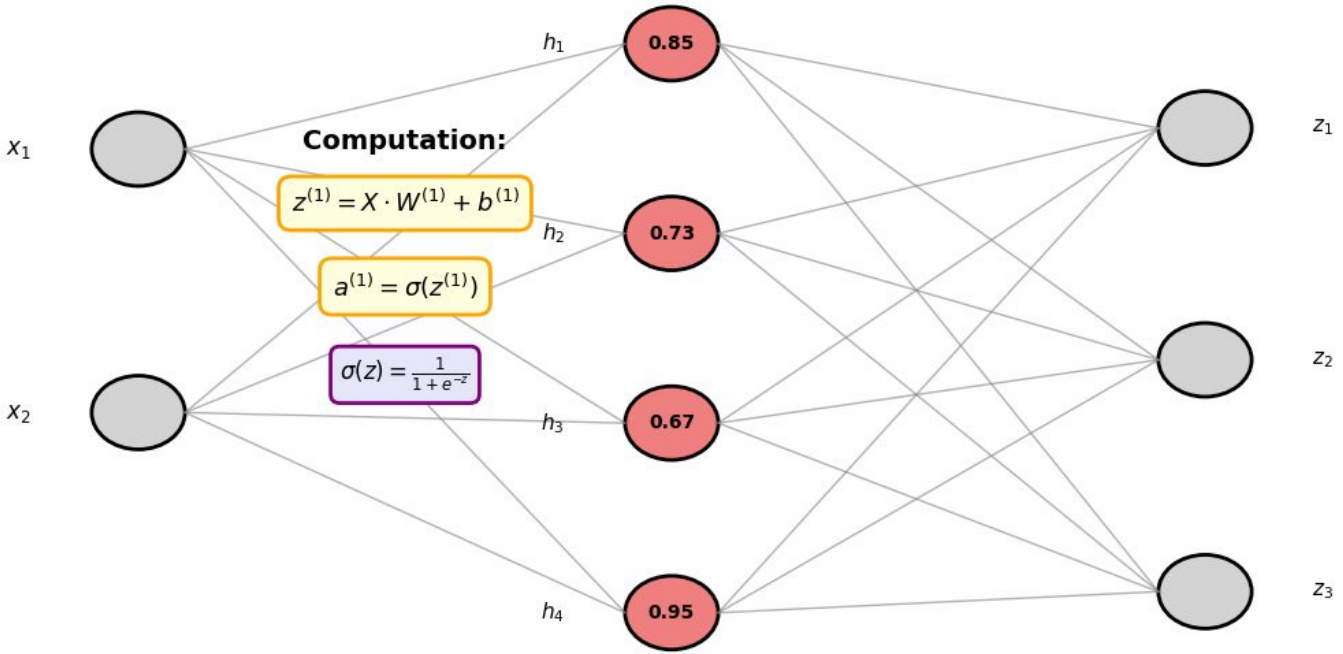


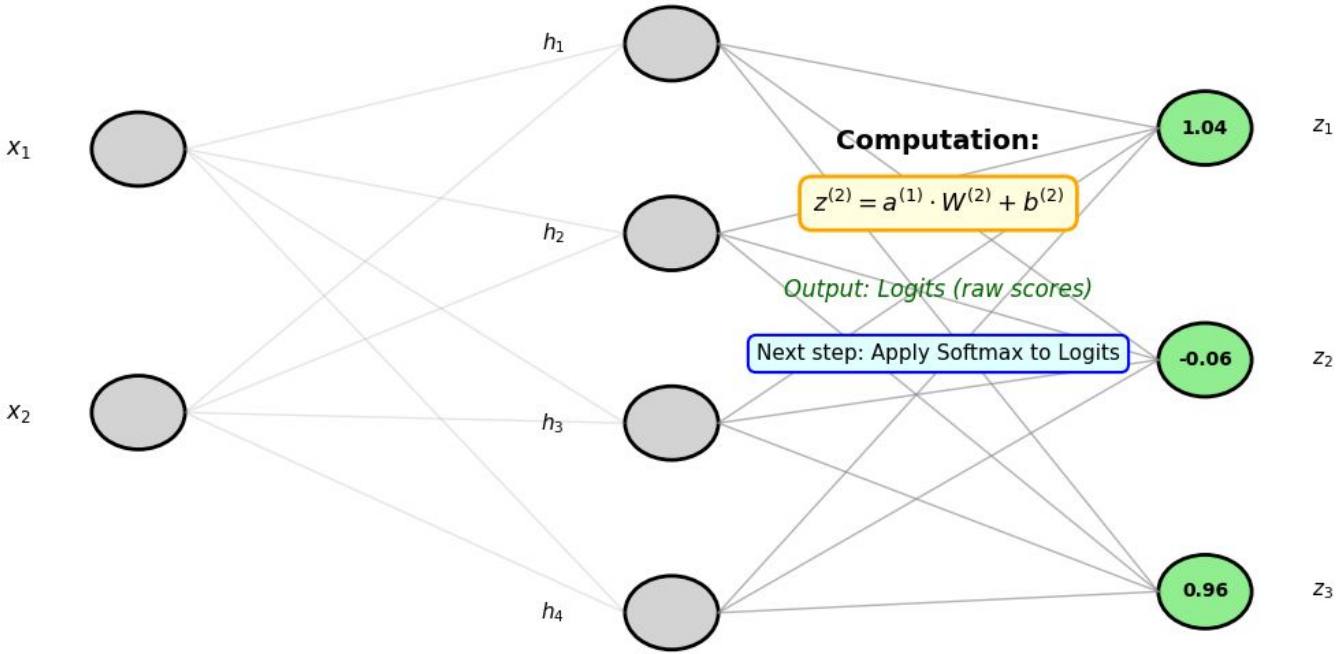
Weight Matrix $W^{(1)}$ (2×4):

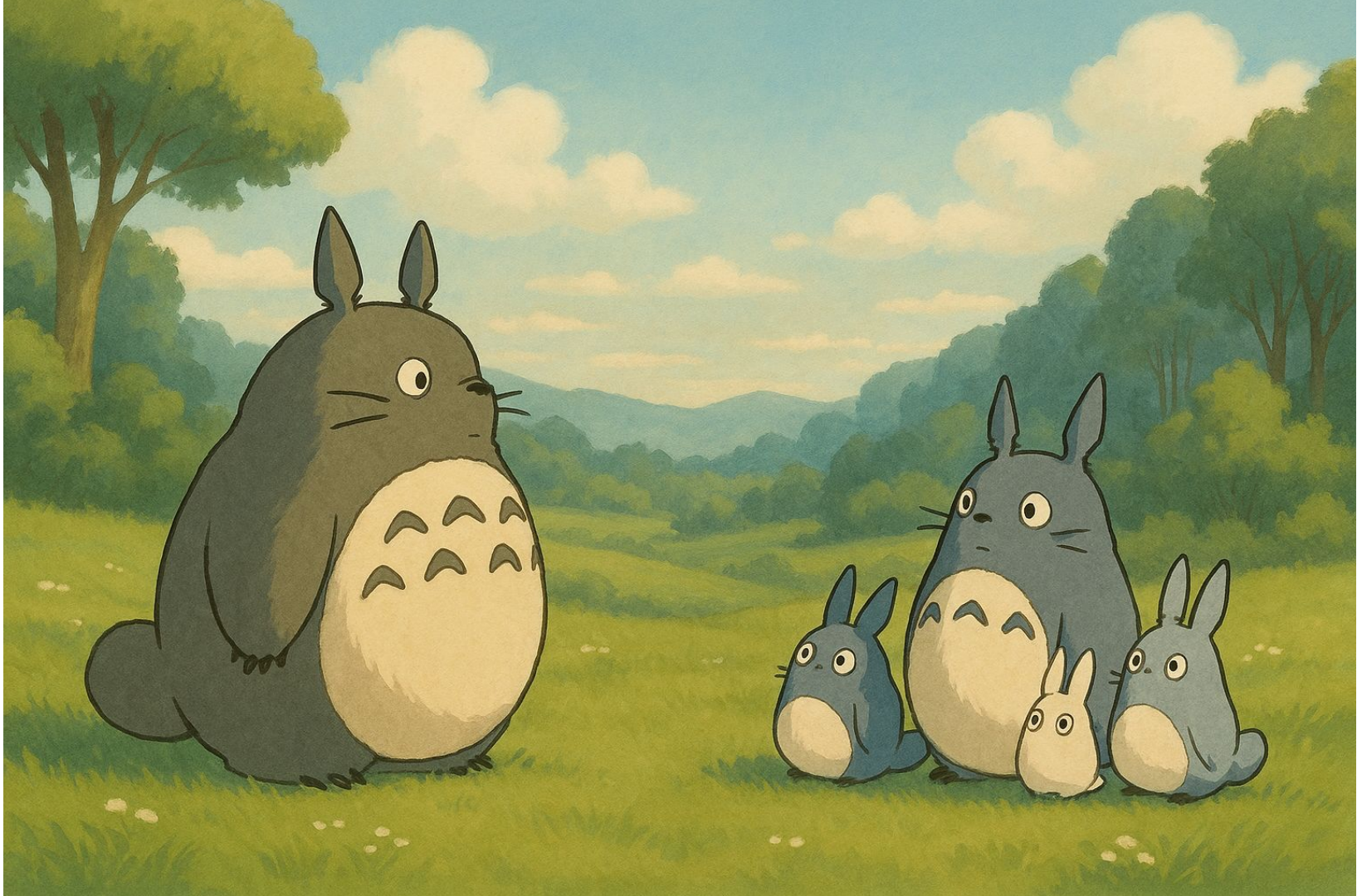
$$\begin{bmatrix} 0.5 & -0.3 & 0.8 & 0.4 \\ 0.2 & 0.6 & -0.4 & 0.7 \end{bmatrix}$$

Bias $b^{(1)}$:

$$[0.1, -0.2, 0.3, 0.1]$$


x_1
x_2
Computation:
$z^{(1)} = X \cdot W^{(1)} + b^{(1)}$
$a^{(1)} = \sigma(z^{(1)})$
$\sigma(z) = \frac{1}{1+e^{-z}}$
h_1
h_2
h_3
h_4
0.85
0.73
0.67
0.95
z_1
z_2
z_3


x_1
x_2
h_1
h_2
h_3
h_4
Computation:
$z^{(2)} = a^{(1)} \cdot W^{(2)} + b^{(2)}$
Output: Logits (raw scores)
Next step: Apply Softmax to Logits
1.04
-0.06
0.96
z_1
z_2
z_3



Training Feedforward Network 🧠



Feedforward (left → right)

- Compute layer activations

Loss function (Distance between prediction and ground-truth)

- Map distance to a scalar error

Backprop (right → left)

- Use the computational graph and chain rule to compute gradients

Loss Functions 📉

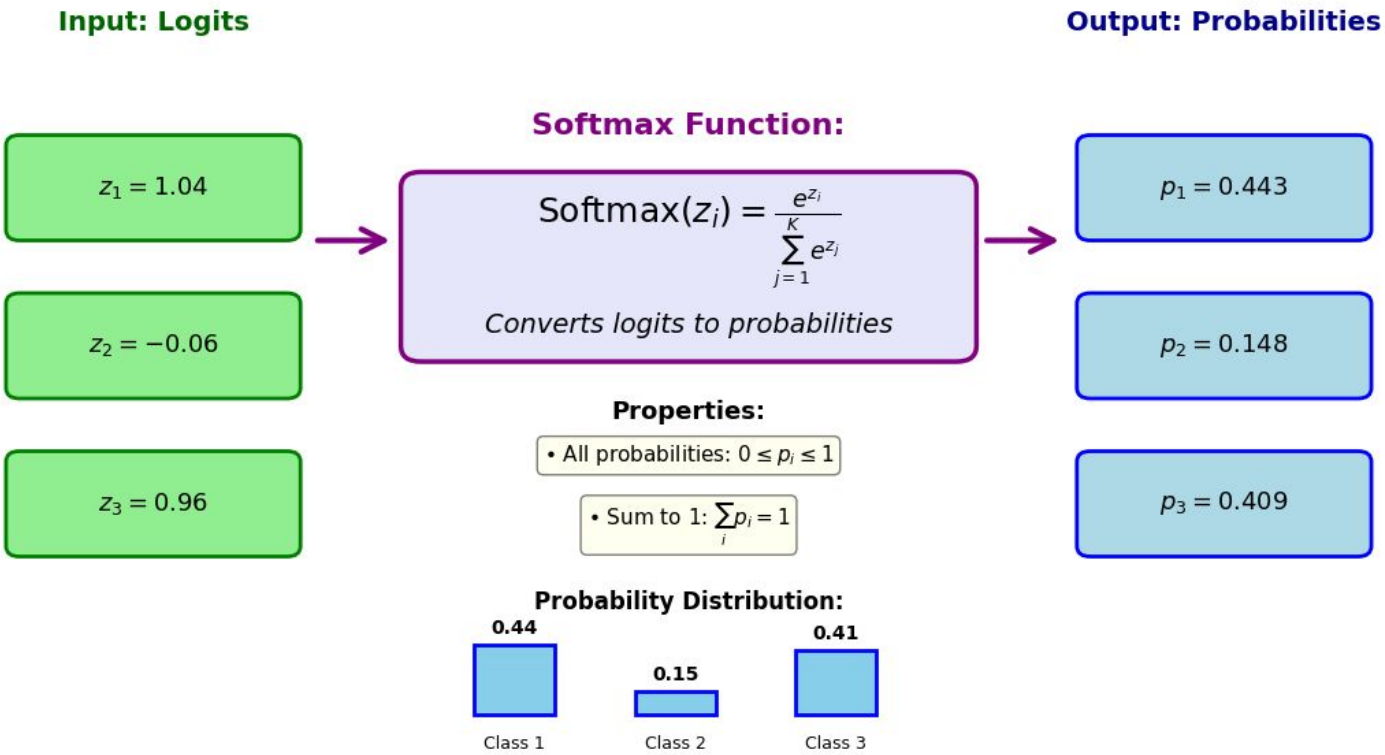


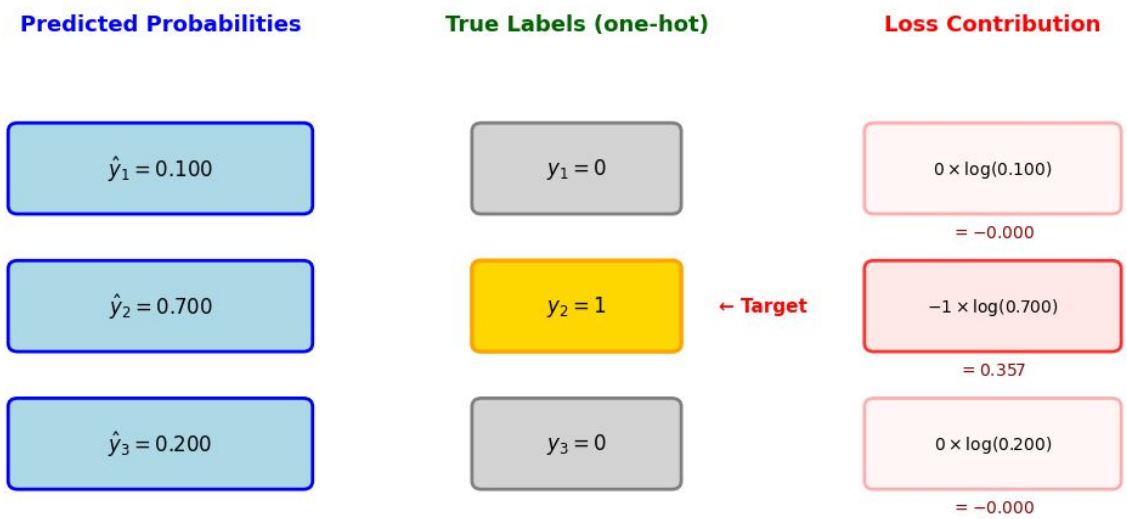
Quantify how far predictions are from targets

Regression → MSE $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Classification → Cross-Entropy $\text{CrossEntropy} = -\sum_{i=1}^{n} y_i \log(\hat{y}_i)$

Lower loss = better model fit on training data


Input: Logits
Output: Probabilities
Softmax Function:
$z_1 = 1.04$
$z_2 = -0.06$
$z_3 = 0.96$
$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
Converts logits to probabilities
$p_1 = 0.443$
$p_2 = 0.148$
$p_3 = 0.409$
Properties:
• All probabilities: $0 \le p_i \le 1$
• Sum to 1: $\sum_i p_i = 1$
Probability Distribution:
0.44
0.15
0.41
Class 1
Class 2
Class 3



$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

Total Loss: $L = 0.3567$

Only the correct class contributes to loss

Training Feedforward Network 🧠



Feedforward (left → right)

- Compute layer activations

Loss function (Distance between prediction and ground-truth)

- Map distance to a scalar error

Backprop (right → left)

- Use the computational graph and chain rule to compute gradients

Backpropagation 🔄



Efficient gradient computation using chain rule

- Backpropagation applies the chain rule to compute derivatives layer by layer

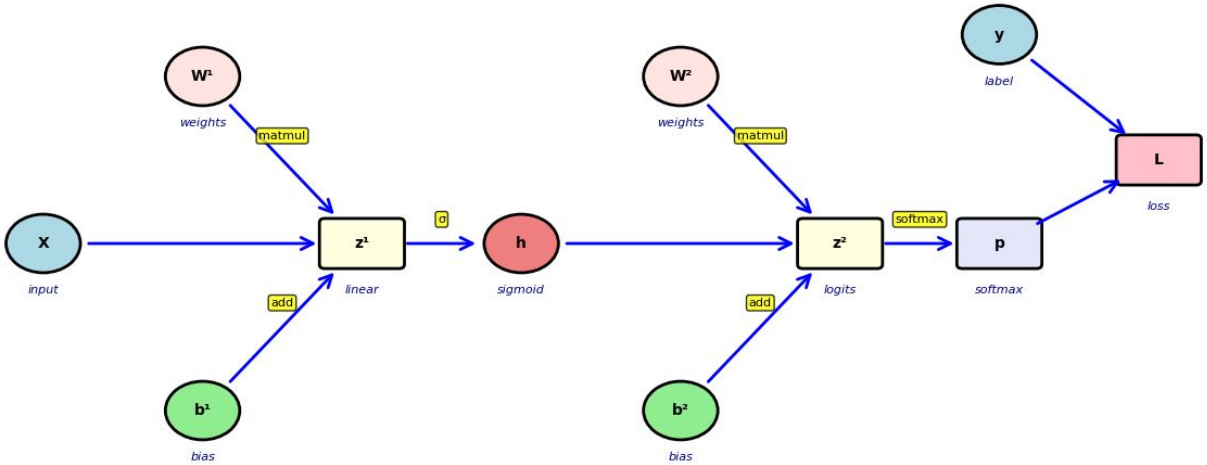
Gradients flow backward from output to input

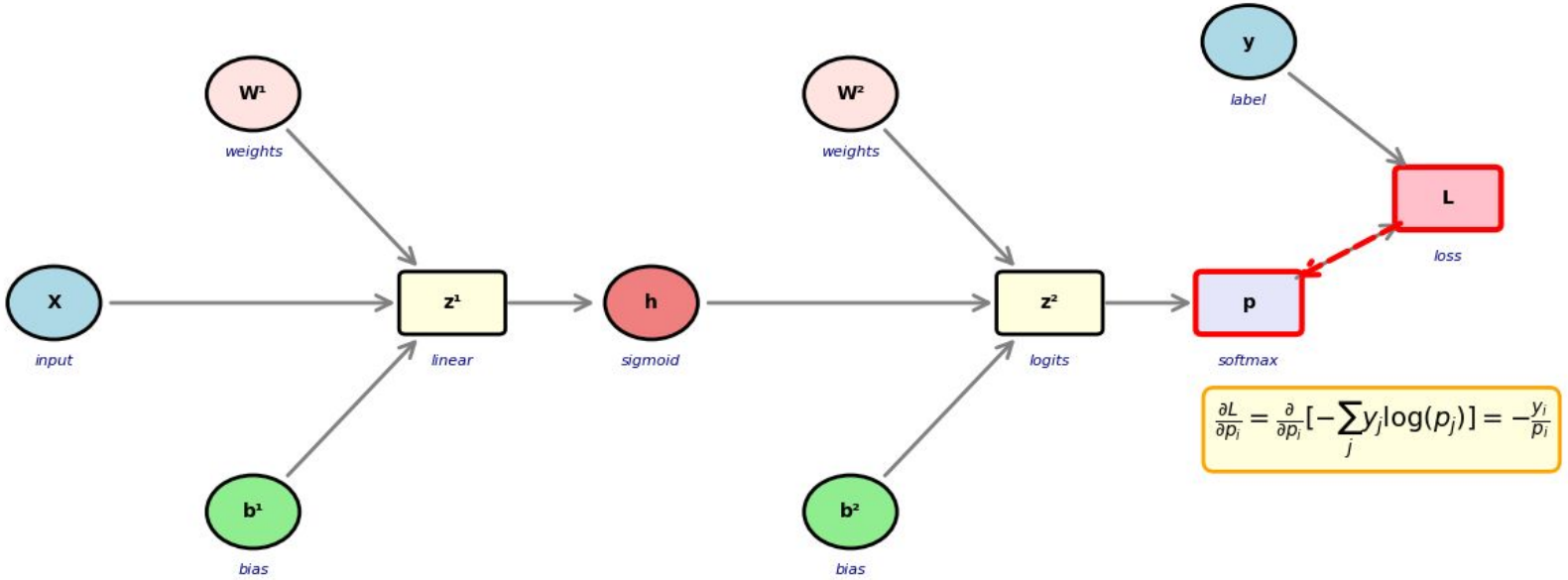
- Each layer receives gradients from the next layer to update its own weights

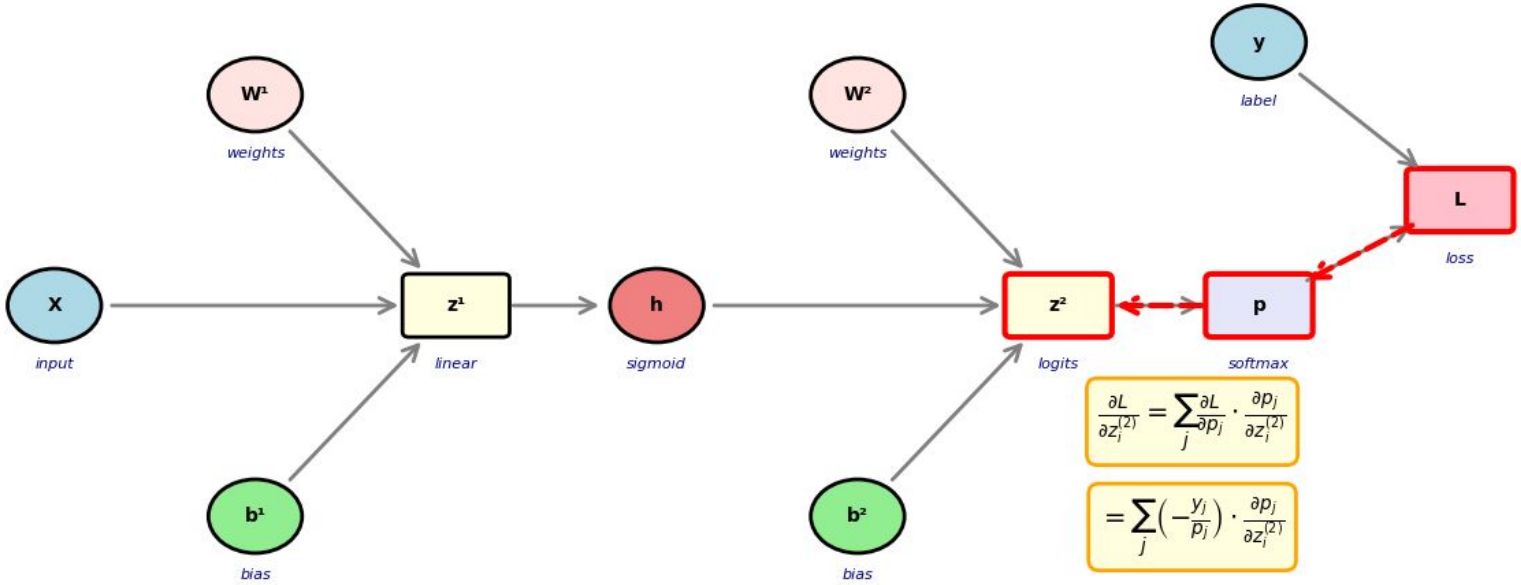
Enables training of deep networks with millions of weights

- Makes large-scale optimization computationally feasible
- Without backpropagation, deep learning would be impossible to train efficiently

Computational Graph




W¹
weights
X
input
z¹
linear
b¹
bias
h
sigmoid
W²
weights
z²
logits
b²
bias
y
label
L
loss
p
softmax
$\frac{\partial L}{\partial p_i} = \frac{\partial}{\partial p_i}[-\sum_j y_j \log(p_j)] = -\frac{y_i}{p_i}$



$$\frac{\partial L}{\partial z_i^{(2)}} = \sum_j \frac{\partial L}{\partial p_j} \cdot \frac{\partial p_j}{\partial z_i^{(2)}}$$

$$= \sum_j \left(-\frac{y_j}{p_j}\right) \cdot \frac{\partial p_j}{\partial z_i^{(2)}}$$

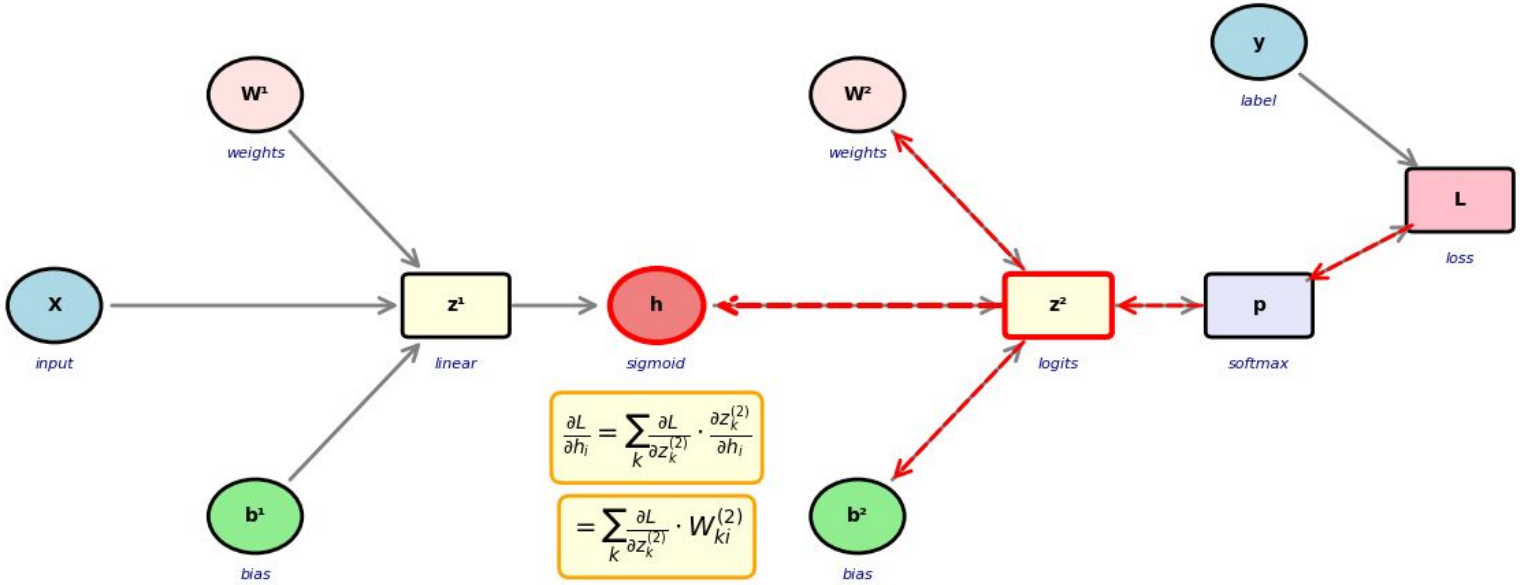
Softmax derivative:

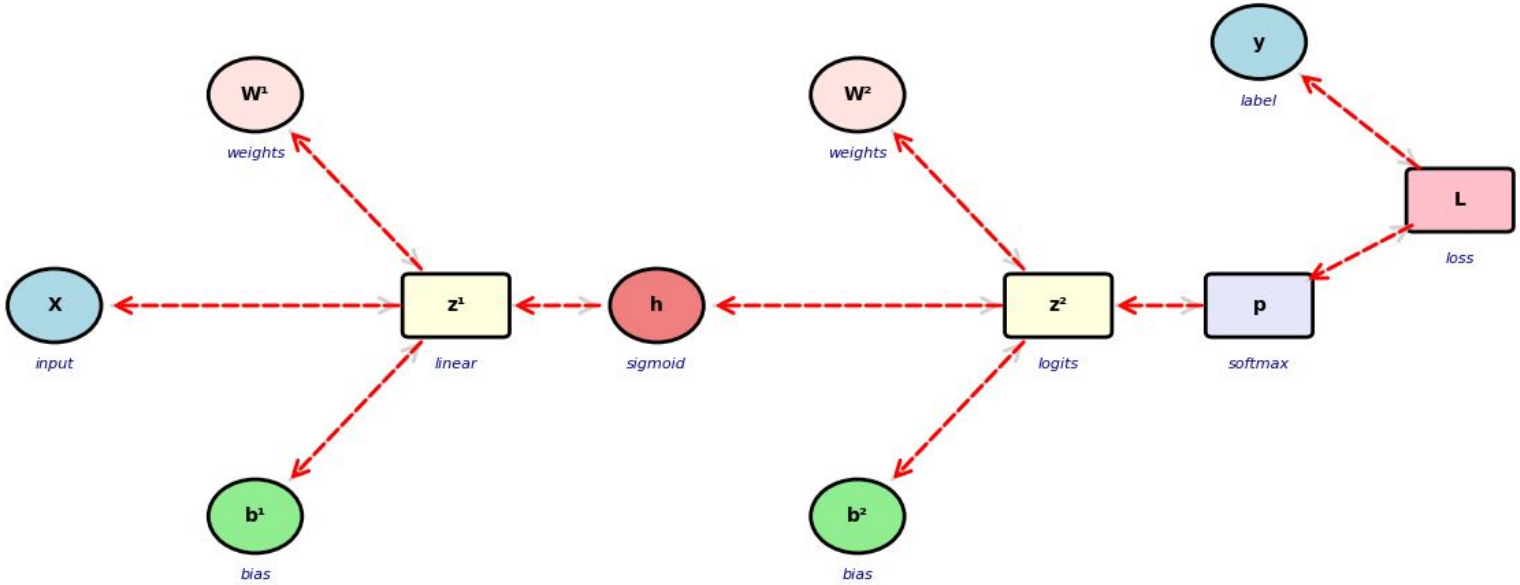
if $i = j$: $\frac{\partial p_i}{\partial z_i} = p_i(1 - p_i)$

if $i \neq j$: $\frac{\partial p_j}{\partial z_i} = -p_i p_j$

Final result:

$$\frac{\partial L}{\partial z_i^{(2)}} = p_i - y_i$$


W¹
weights
X
input
z¹
linear
h
sigmoid
b¹
bias
W²
weights
z²
logits
b²
bias
y
label
L
loss
p
softmax
$\frac{\partial L}{\partial h_i} = \sum_k \frac{\partial L}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial h_i}$
$= \sum_k \frac{\partial L}{\partial z_k^{(2)}} \cdot W_{ki}^{(2)}$



Backpropagation Complete!

✓ Backward Pass: Compute all gradients (red dashed)

✓ Gradients computed via chain rule recursively

✓ Update: $\theta \leftarrow \theta - \eta \cdot \partial L/\partial \theta$ (Gradient Descent)

Vanishing & Exploding Gradients ⚠️



Backprop multiplies one derivative per layer (chain rule)

- Sigmoid′ ≤ 0.25 (Derivatives Comparison) → 10 layers: $0.25^{10} \approx 0.000001$
- Early layers get almost no gradient and stop learning: vanishing
- Factors > 1 compound the other way → huge steps, NaN: exploding

The standard fixes

- ReLU / GELU: derivative = 1 for positive inputs
- He / Xavier initialization, normalization layers (BatchNorm, LayerNorm)
- Residual connections (next week) and gradient clipping

Optimizers ⚙️



Stochastic GD $\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$

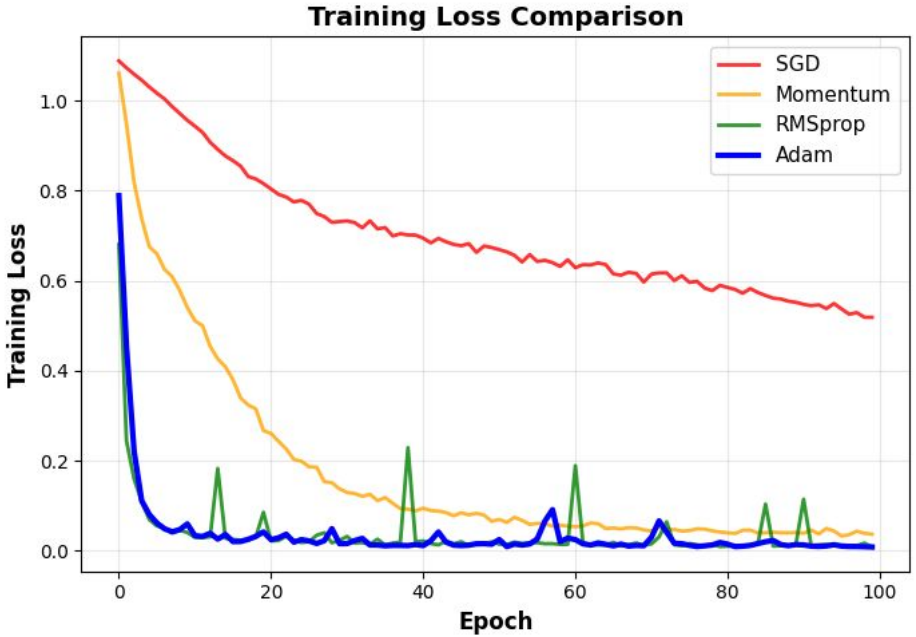
Too small → slow 🐢 too large → divergence 😳

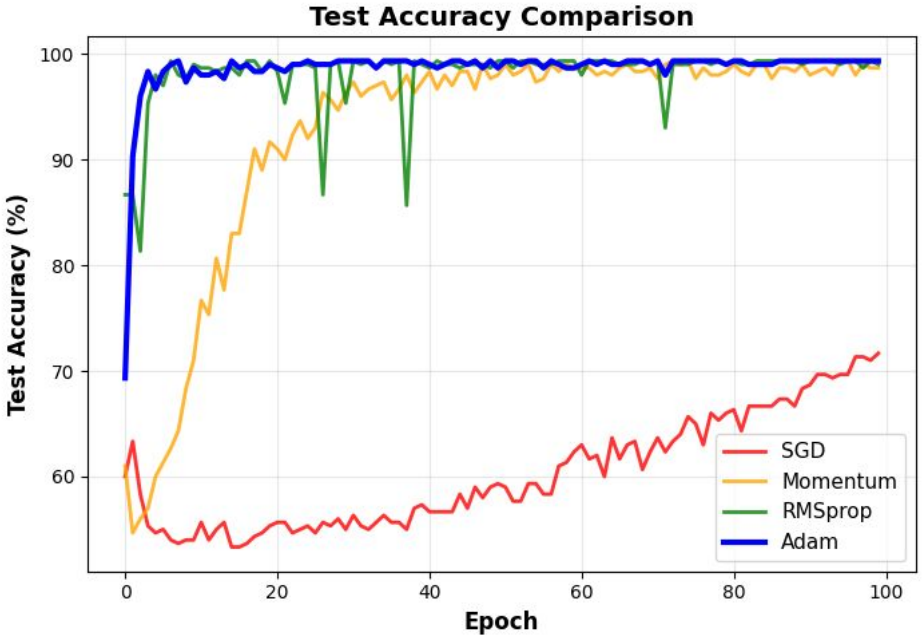
Momentum → Accelerates learning by smoothing gradients with past updates

RMSProp → Adapts learning rates based on recent gradient magnitudes

Adam → Momentum + RMSProp

Comparisons





🎤 Mock Interview



Interviewer: “Your MLP’s training loss drops for 200 steps, then shoots up and turns into NaN. What do you try first?”

A) Add more hidden layers
B) Lower the learning rate
C) Train for more epochs
D) Switch ReLU to sigmoid

30 seconds, no talking. Then raise your hand for one letter.



Regularization

Neural Networks Overfit Too 😵

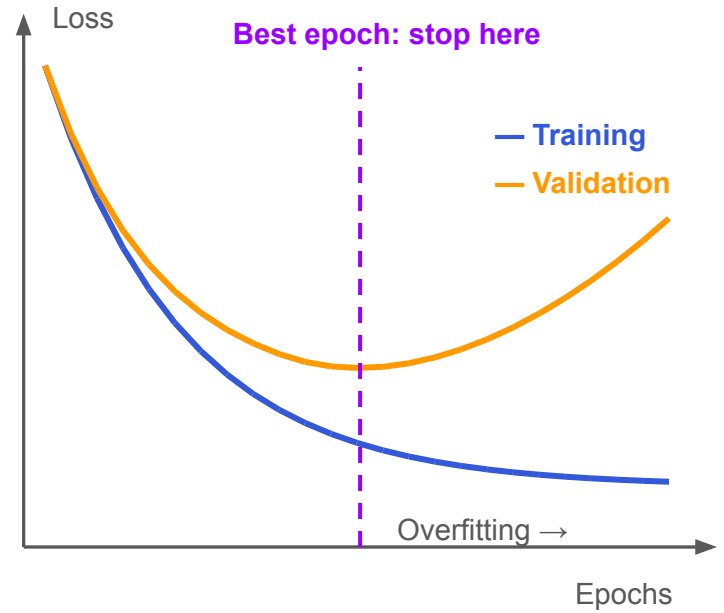
Cause → millions of weights memorize

Symptom → validation loss turns up

Diagnosis → high variance (Week 4)

Fixes → dropout, early stopping

Also → weight decay = L2 / Ridge



Dropout 🎲

Training → randomly drop neurons

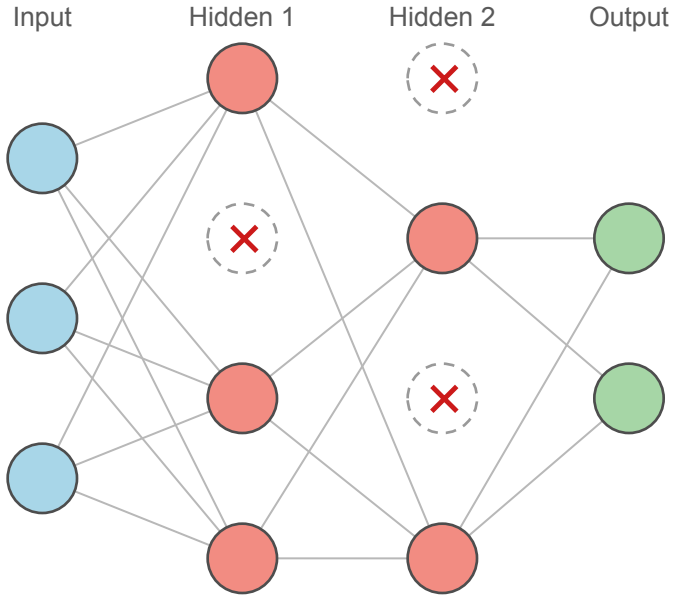
Dropout rate → p = 0.1–0.5

Effect → no single neuron to lean on

Ensemble view → bagging (Week 5)

Test time → keep every neuron

PyTorch → model.train() / .eval()



Each batch drops a different random subset

Early Stopping ⏸️



Evaluate validation loss after every epoch

Keep the best checkpoint, not the last one

Stop when validation has not improved for k epochs (patience, e.g. 5)

- Like capping a tree's depth (Week 5): stop growing before it memorizes

Free, and it stacks with weight decay and dropout

🎤 Mock Interview

Interviewer: “Your MLP hits 99% training accuracy but 72% on validation. Validation loss has been rising since epoch 10 of 50. What do you do first?”

A) Train 50 more epochs
B) Restore the epoch-10 checkpoint, add dropout / weight decay
C) Increase the learning rate
D) Add more layers

30 seconds, no talking. Then raise your hand for one letter.

Where MLPs Still Run in Industry 🏭



Recommendation & Ads ranking

- Meta DLRM, YouTube, Google Wide & Deep: user / item embeddings → MLP → click or watch-time score

Two-tower retrieval

- User tower + item tower (each an MLP) → dot product → top candidates out of millions (Week 3 funnel)

Inside every LLM

- The feed-forward block in each Transformer layer is an MLP (AI of the Week)

Glue between models

- Small MLP heads on embeddings: classifiers, LLaVA's vision-to-language projector

MLP Tricks That Ship 🚀



Embed every ID and category

- user_id, zip code, job title → learned vectors: fixes Week 5's zip-code problem

Hash IDs into a fixed number of buckets

- New users and items still get an embedding; the table size stays bounded

Short, funnel-shaped towers

- YouTube's ranker: 1024 → 512 → 256 (ReLU); width and depth are capped by serving cost

Shrink for serving

- Embedding tables are most of the model → store them in 8 bits; distill big → small to fit the 10 ms budget (Week 3)

From MLP to Transformer ➡️



A Transformer block = Attention + MLP

- Attention: tokens exchange information with each other
- MLP: transforms each token on its own (today)
- Both wrapped with residual connections + LayerNorm

Everything from today carries over

- GELU, backprop, Adam, dropout, weight decay

Next week, Transformers in Practice: the one new piece is attention

Recap 🧭



Start from linear models → limited expressiveness

Add nonlinear activations → learn complex functions

Stack layers → Multilayer Perceptron (MLP)

Train by forward + backward passes to minimize loss

Tune learning with optimization and regularization

Forms the foundation for CNNs, Transformers and ChatGPTs